

Auto Fine-Tuning GPU Parameters for AI Workloads

Akhil Ivaturi¹, Kesav Nagendra², Harshith Kantamneni³, Sai Aasrith Avvaru⁴

¹ aivaturi@wisc.edu

² nagendra2@wisc.edu

³ hkantamneni2@wisc.edu

⁴ savvaru2@wisc.edu

Abstract

Suboptimal performance may result from the use of static, one-size-fits-all kernel launch parameters in machine learning (ML) workloads that are executed on GPUs. The issue is resolved by this project, which autonomously optimizes GPU parameters (specifically CUDA kernel launch configurations) to more effectively align with the requirements of various AI workloads. We create a custom GPU kernel and integrate it with PyTorch. Subsequently, we gather performance data for a variety of thread block configurations. The selection of an optimal thread block size for each scenario is facilitated by the training of a neural network model to determine kernel runtime based on the workload and launch parameters. The optimized kernel is integrated into the training loop of a neural network to validate our methodology. The results indicate that an ML model that employs the auto-tuned GPU kernel accomplishes a faster training process (a total training time reduction of over one second) than a baseline with static settings, while maintaining the accuracy of the model. These results illustrate the potential of dynamic GPU parameter tuning to enhance the efficiency of AI training, and we propose future directions for the expansion of this approach to more generalized environments.

Index Terms: Pytorch, Kernal, Neural Network.

1 Introduction

In machine learning, graphics processing units (GPUs) have become essential for expediting inference and training. Nevertheless, the efficient utilization of GPUs necessitates the meticulous adjustment of low-level execution parameters. When launching CUDA kernels for neural network operations, developers typically employ static GPU parameters, such as a fixed block size and the number of threads per block. Although fixed defaults are acceptable in most situations, they are inadequate for accommodating fluctuations in duties. Different models, layers, or dataset sizes may have distinctive optimal configurations; a kernel launch configuration that is optimal for one scenario may be suboptimal for another. Therefore, the utilization of the same thread block dimensions and other launch parameters in all training tasks may impede performance, as certain GPU resources may be underutilized or memory access patterns may not be optimal. This can result in GPU computations running at a slower pace than is required for certain duties, thereby squandering valuable computing time and energy.

This inefficiency is the impetus for the auto-tuning of GPU parameters in ML applications. The potential for improved memory throughput, enhanced GPU core occupancy, and overall faster execution can be achieved by dynamically adjusting kernel launch parameters to the characteristics of a given workload. Previous research in high-performance computing has observed that GPU utilization is frequently significantly lower than peak capabilities when generic configurations are employed. In the context of deep learning, where training entails repetitive kernel executions (for matrix multiplications, convolutions, etc.), even minor inefficiencies per iteration can accumulate to cause substantial slowdowns. This project aims to address this gap by developing a system that autonomously selects the most appropriate settings for new workloads by recognizing the correlation between GPU launch configurations and runtime performance for a specific operation. In

essence, our objective is to ensure that GPU kernel deployments are self-tuning and workload-aware.

Our method of automated GPU parameter optimization for AI workloads is detailed below. First, we conduct a review of pertinent research that has addressed comparable GPU performance optimization challenges. Subsequently, we provide an overview of CUDA programming, including the kernel launch parameters and their impact on performance, as well as the process of incorporating custom GPU kernels into the PyTorch ML framework. We then provide a comprehensive explanation of our methodology, which includes the development of a custom CUDA kernel and the collection of performance data. Subsequently, we construct a prediction model that determines the optimal thread block configuration at runtime and integrates it into a neural network model. We assess the performance enhancements that our methodology achieves in comparison to a baseline and engage in a discussion regarding the findings. We conclude by outlining future work directions and providing critical takeaways for the expansion and generalization of this auto-tuning strategy.

2 Background

2.1 CUDA Programming Model and Kernel Launch Parameters

NVIDIA's CUDA programming paradigm enables developers to create parallel GPU functions, which are referred to as kernels. The programmer is required to specify the number of threads that will execute a CUDA kernel and the organization of those threads when it is launched. This is accomplished by selecting a grid size (the number of thread blocks to launch) and a block size (the number of threads per block). Threads within a block execute on the same Streaming Multiprocessor and can collaborate through shared memory, whereas distinct blocks operate independently (possibly concurrently on different SMs). A kernel launch is a grid that is composed of the aggregate of all blocks. The thread hierarchy of CUDA is as follows: The GPU executes

kernels by initiating a multitude of threads that are grouped into blocks, which are subsequently organized into a grid. A set of threads that execute the same kernel code is present in each block (Block-0, Block-1,..., Block-(K-1) as illustrated). The GPU’s streaming multiprocessors (SMs) distribute the grid of units. Blocks are executed independently, while threads within the same block can synchronize and share data through rapid on-chip memory. Selecting the appropriate number of threads per block (and their arrangement in the x, y, and z dimensions) is crucial for performance, as it influences the efficient utilization of the GPU’s cores and the organization of memory accesses.

The maximum number of threads per block is restricted by hardware, which is a critical constraint in CUDA. The maximum number of threads in a single block is currently set at 1024 by NVIDIA GPUs. This implies that the sum of the block’s dimensions ($\text{blockDim.x} * \text{blockDim.y} * \text{blockDim.z}$) cannot exceed 1024. For example, a block of size (32,32,1) employs 1024 threads, which is permissible; however, (33,32,1) would surpass this limit and thus be prohibited. Additionally, block dimensions are frequently selected to be multiples of the warp size (32 threads) in order to optimize performance. However, the optimal shape (e.g., $32 \times 1 \times 1$ vs $16 \times 2 \times 1$ vs $8 \times 4 \times 1$, all of which generate 32 threads) can differ depending on the kernel’s access patterns. The number of blocks that can operate concurrently on an SM is restricted, and the overall occupancy is reduced, as a block that is too large may consume an excessive amount of resources (registers, shared memory) per block. If it is too small, it may result in insufficient parallel work to conceal memory latency. Consequently, the selection of block size and grid size is a performance optimization exercise that involves the balance of occupancy (enough active threads to fully utilize the GPU) against per-thread resource availability and overhead.

Another pertinent concept is that GPU kernels frequently possess empirically optimal launch configurations that are contingent upon the size of the data. For example, a matrix multiplication kernel may operate the most efficiently with a specific block width and height (blockDim.x , blockDim.y) that optimizes the utilization of memory caches and computation units. Developers either manually benchmark a few configurations or rely on general heuristics in practice. Our project employs an automated approach, which involves meticulously benchmarking a variety of configurations for a specific kernel and utilizing a model to predict the configuration that minimizes runtime for new inputs.

2.2 Integration of Custom CUDA Kernels with PyTorch

In order to implement custom GPU optimization in the context of machine learning, we incorporate our CUDA kernel into the PyTorch framework. PyTorch typically offers high-level operations, such as matrix multiplication or the linear layer, that internally utilize optimized libraries (such as cuBLAS) with their own predetermined launch settings. By developing a customized CUDA extension, we circumvent this limitation. Custom C++ and CUDA code can be written by users and compiled into a module that can be imported into Python using PyTorch’s extension interface. Our kernel was constructed using the `torch.utils.cpp_extension` utilities. One can specify CUDA source files (`.cu`) and C++ source files in a setup script by utilizing `CUDAExtension`. The PyTorch build system will compile them

into a Python-callable shared library. This mechanism abstracts away a significant amount of the boilerplate, allowing for the seamless invocation of the custom CUDA kernel as if it were a standard PyTorch function.

We developed a custom CUDA kernel for matrix multiplication and encapsulated it as a PyTorch custom operator in our implementation. This entailed the development of a CUDA kernel function (designated as `__global__`) that executes batched matrix-matrix multiplication. Additionally, a C++ “glue” function was implemented, which PyTorch invokes to initiate the CUDA kernel with the specified block and grid dimensions. PyTorch’s JIT compilation facility was employed to compile this, enabling PyTorch to load the compiled GPU code at runtime. The custom kernel can be invoked on tensors within PyTorch models after it has been integrated, utilizing the framework’s autodiff (we either provided or relied on PyTorch’s automated differentiation for gradients, treating the custom operation as atomic). The integration was challenging due to environmental factors. For instance, we encountered build failures when attempting to match the CUDA toolset and GCC compiler versions on the HPC system. Consequently, we had to modify our build environment to be compatible with the cluster’s configuration. After these issues were resolved, the outcome was a PyTorch layer that was supported by our own CUDA kernel, allowing us to exert precise control over its launch parameters.

3 Literature Review

Several prior works have explored automated tuning of GPU kernel parameters and dynamic optimization of GPU workload execution. We discuss a selection of relevant approaches below:

3.1 KLARAPTOR

Kernel LAunch parameters A tool known as RAtional Program estimaTOR (KLARAPTOR) dynamically determines the optimal CUDA kernel launch parameters by utilizing performance prediction models. It leverages an LLVM compiler pass and NVIDIA’s CUPTI profiling API to incorporate a “rational” program that utilizes hardware information and input data to predict and select the optimal thread block and grid configuration for a specific kernel at runtime. This method efficiently automates a manual tuning process by adjusting kernel launch dimensions to achieve near-optimal performance for each execution.

3.2 Machine Learning for CFD Autotuning

To optimize GPU kernel parameters in computational fluid dynamics (CFD) applications, Xue et al. implemented a machine learning-based autotuning technique. The method they employ employs fully connected neural networks to simulate the relationship between the resulting execution time and the GPU launch parameters (14 important parameters, including the number of thread blocks and threads per block). Their autotuner is capable of predicting the execution time for various parameter configurations by training on sample trials. This resulted in substantial performance improvements for CFD simulations, despite the fact that only a small portion of the search space needed to be sampled. This serves as an illustration of the effectiveness of neural network models in GPU performance optimization.

3.3 LiveTune (Dynamic Parameter Tuning):

Shabgahi et al. developed LiveTune, a framework that enables the dynamic adjustment of parameters during feedback-driven optimization processes (e.g., ML model training) without disrupting the program. Although it is not specific to CUDA kernels, it is designed to address hyperparameters in learning algorithms, allowing for real-time tuning based on feedback (e.g., modifying the learning rate or other settings on the fly). In training loops, real-time parameter adjustment can save up to 60 seconds of runtime and considerable energy per hyperparameter change, according to LiveTune’s evaluations. Our dynamic approach is motivated by the concept of continuous, runtime adaptation of parameters for efficiency, although we use it to modify low-level GPU launch settings rather than algorithmic hyperparameters.

3.4 Slate (Workload-Aware Multiprocessing):

Slate is a system that is capable of administering multiple concurrent GPU workloads in a workload-aware manner. Allen et al. (IPDPS 2019) introduced Slate, which enables multiple processes to share a GPU by concurrently scheduling their kernels when their resource requirements are complementary. Slate enhances GPU utilization by dynamically scaling kernels and spatially sharing the GPU through software. In comparison to NVIDIA’s default multiprocess service, Slate demonstrated an average improvement in system throughput of approximately 11% (and as high as 35% in the most extreme cases) through experiments. Slate emphasizes the potential benefits of GPU execution that are customized to the characteristics of the workload, despite its emphasis on multi-application scheduling rather than the optimization of a single kernel’s parameters.

3.5 Aker (Adaptive Kernel Fusion):

Aker is a method that seeks to optimize GPU utilization by combining kernels and scheduling them intelligently on streaming multiprocessors (SMs). It utilizes a performance model (which includes a duration predictor) to schedule these fused kernels while ensuring quality-of-service for critical tasks, and it statically merges the code of small GPU kernels to enable them to execute together on an SM. Experimental results suggest that Aker can increase the throughput of best-effort GPU applications by approximately 50% on average compared to state-of-the-art methods, while maintaining the performance of high-priority tasks. This illustrates an additional aspect of GPU autotuning: Aker modifies the kernel code execution structure rather than modifying launch parameters. However, it is similarly motivated by the necessity to more effectively utilize GPU parallel resources in accordance with the workload balance.

In conclusion, previous research has addressed GPU performance tuning from a variety of perspectives, including predictive modeling of launch parameters (KLARAPTOR, ML-based autotuners), dynamic runtime adjustment frameworks (LiveTune), and systems-level scheduling and fusion (Slate, AkerTM). Our project distinguishes itself by integrating the autotuning mechanism directly into the training process of an ML model and by concentrating on the use of machine learning to autotune kernel launch parameters for deep learning workloads at runtime. This work is inspired by the aforementioned works: we employ predictive

modeling, such as KLARAPTOR and the CFD autotuner, dynamic adjustment in-process analogous to LiveTune, and strive for improved resource usage per kernel similar to Slate and Aker, but at the level of an individual network’s operations.

4 Methods

4.1 Custom CUDA Kernel Development

The initial phase involved the creation of a custom GPU kernel that could be customized. We selected a matrix multiplication kernel as a representative operation due to the computationally intensive nature of matrix multiplies, which are prevalent in neural network layers (e.g., the linear layers of an MLP). The implementation of our CUDA kernel was designed to multiply two matrices (or bundles of matrices) of the appropriate dimensions. Our focus was on the implementation of two-dimensional thread blocking. Each thread block is configured with dimensions (`blockDim.x`, `blockDim.y`) that correspond to a specific number of output matrix elements computed by the block in the `x` (column) and `y` (row) directions. This layout was chosen to enable us to experiment with the `x`-size and `y`-size of the block as tunable parameters. The kernel accumulates over the shared dimension by utilizing each thread to calculate one element (or a tile) of the output matrix.

We integrated the CUDA kernel with PyTorch using the `CUDAExtension`, as previously described, after writing and verifying its accuracy. This enabled Python to access the kernel. We developed a PyTorch Function/Module wrapper that can be invoked with a specific (`block_x`, `block_y`) configuration. In actuality, the kernel is launched by the call from Python to our custom op, with a grid size that is determined by the input matrix size and the selected block dimensions. We verified that the kernel launch adhered to CUDA limits (e.g., if `block_x × block_y` exceeds 1024, we consider the configuration invalid). This integration subsequently enabled us to utilize the kernel in an end-to-end model training cycle for evaluation.

4.2 Benchmarking and Data Collection

Next, the custom kernel was benchmarked across a range of GPU launch configurations and input sizes to collect performance data. Experiments were scripted to execute the kernel with varying block size configurations. In particular, we systematically varied `blockDim.x` and `blockDim.y` across a spectrum of values. For instance, we selected values for `block_x` (thread count in the `x` dimension) and `block_y` (thread count in the `y` dimension) that do not exceed 1024. This encompassed a variety of configurations, such as (8,8), (16,16), (32,2), (32,4), (64,1), and so forth, which included both tiny and large blocks, as well as 1D and 2D shapes. Additionally, we varied the magnitude of the problem, which in this instance could be the matrix dimensions or the batch size of matrices being multiplied. In particular, we conducted experiments with varying matrix sizes (or batch sizes of a fixed-size matrix) to simulate varying workload sizes, as the optimal block configuration may be contingent upon the data size. The kernel’s execution time was recorded for each configuration, which was a combination of `block_x`, `block_y`, and input capacity. The kernel launch was timed by synchronizing the GPU and measuring the wall-clock time (possibly averaged over numerous runs for stability).

A dataset of the form: $(\text{input_size}, \text{block_x}, \text{block_y}) \rightarrow \text{runtime}$ was generated as a result of this exhaustive benchmarking. This data was subsequently preprocessed and filtered. Any configuration that violated constraints (such as $\text{block_x} \times \text{block_y} > 1024$, which we expressly skipped during collection) or caused kernel errors was eliminated. Additionally, we observed that a certain number of the configurations produced runtimes that were exceedingly similar, suggesting that the addition of more threads may not have been beneficial beyond a certain threshold. In total, we assembled a dataset that was diverse; however, we discovered that certain features were deficient in diversity (e.g., certain block_x ranges may not have been entirely represented with varying input sizes). Consequently, the unprocessed data necessitated some augmentation and cleaning, as will be explained in the following section.

4.3 Feature Engineering and Performance Modeling

Our objective was to develop a machine learning model that could forecast the kernel’s duration in response to a specific configuration and input size by utilizing the benchmark data that was collected. This predictive model would function as a surrogate for the actual GPU execution. It would enable us to rapidly determine the speed at which the kernel would operate with various thread block settings, thereby eliminating the need to conduct a brute-force test each time. In consideration of the success of previous studies that employed neural networks to predict performance, we selected a neural network model for this regression task.

In order to enhance the model’s learning capabilities prior to training, we implemented feature engineering. We constructed derived features that appeared to be pertinent to runtime, rather than explicitly feeding raw parameters (input_size , block_x , block_y). For instance, we incorporated the total number of threads per block ($\text{block_x} \times \text{block_y}$) as a feature, as this determines the amount of parallel work a block performs. Another characteristic was the ratio of work per thread in a single dimension, such as $\text{input_size} / \text{block_y}$ (if block_y is interpreted as covering the output rows, this ratio would indicate the number of blocks required for the specified input size in the y dimension, which may be correlated with memory access patterns). Additionally, we evaluated the proportions of block_x and block_y . Subsequently, all features were normalized (scaled) due to the potential for significant variation in raw values, particularly input quantities, and the fact that neural networks perform better when trained on normalized inputs. We discovered that feature scaling was essential, as the model initially encountered difficulties as a result of the data’s varying ranges. This was a component of the challenge, as the dataset had limited diversity and necessitated meticulous scaling to prevent the model from becoming stymied.

Our neural network model was a multilayer perceptron (MLP) regressor that was relatively straightforward. We reserved a portion of the data we collected for validation and allocated a portion for training. The model acquired the ability to transfer the input features (block configuration and size) to the output (predicted runtime) during the training process. At first, we

had contemplated expressing this as a direct optimization or classification problem (in order to directly predict the optimal block_x and block_y). However, this proved to be challenging due to the discrete and combinatorial nature of the output and the scarcity of data for each discrete class. Instead, regression, which is a simpler learning task with limited data, could be employed by transitioning to predicting runtime as a continuous value. The process of identifying the optimal configuration for a new input size is straightforward once the model has been trained. We can enumerate candidate (block_x , block_y) pairs, route them through the model to obtain predicted runtimes, and select the configuration with the lowest predicted runtime.

We assessed the MLP’s prediction accuracy on the validation set after it had been trained to ensure that it was accurately capturing the performance trends. We observed that the model effectively learned the coarse behavior (e.g., it was able to identify which configurations are typically quicker), despite the possibility of some error in the precise milliseconds of runtime prediction. This level of precision is adequate for our objective of selecting an optimal configuration. The argmin of the model’s runtime prediction is the configuration that is selected for a specific input. The model consistently predicted an optimal block configuration of $\text{BLOCK_X} = 32$, $\text{BLOCK_Y} = 2$ for the first layer’s matrix multiply in our test scenario, which involved a specific range of matrix sizes corresponding to the neural network we employ later. This is particularly noteworthy. The predicted optimum, which we will subsequently verify in our experiments, was this configuration (32×2 threads, i.e., 64 threads per block).

4.4 Custom Layer Integration into an MLP

In order to illustrate the advantages of our auto-tuning approach in a genuine ML workload, we incorporated the aforementioned mechanism into a basic neural network model. We concentrated on a basic Multi-Layer Perceptron (MLP) model for a classification task. The MLP is composed of an input layer (a fully-connected linear layer), several concealed layers, and an output layer. We replaced the initial linear layer of this network with a custom layer that employs our optimized CUDA kernel. When this custom layer is invoked, it employs the trained prediction model to determine the thread block dimensions for the current input batch. Subsequently, it initiates the matrix multiplication kernel with these dimensions for $\text{input_data} \times \text{weights}$ multiplication.

In particular, the custom layer receives input data of shape $(\text{batch_size}, \text{input_dim})$ and weight parameters of shape $(\text{input_dim}, \text{output_dim})$ during each forward pass of training. We query our prediction model to determine the optimal (block_x , block_y) for the given batch size (or a representation of the input dimensions). Next, we invoke the CUDA kernel by utilizing the predicted block configuration in the launch parameters and passing in the data and weights through our PyTorch extension. Ideally, the kernel computes the same result as a standard PyTorch Linear layer, but it does so at a quicker rate as a result of an optimized launch configuration. The network’s remaining components, including subsequent layers and loss computation, remain unaltered. The implementation of backward propagation support was straightforward. For the purposes

of this project, we permitted PyTorch’s autograd to estimate gradients through the custom operation, treating it as a black box differentiable function. This is due to the fact that it is essentially linear matrix multiplication, which autograd can handle by backprop through the inputs and weights.

We then trained this custom-augmented MLP on a sample dataset (e.g., a small image classification or synthetic dataset) and compared its performance to a baseline MLP that was identical except for the first layer, which was the standard PyTorch implementation.. In order to attribute any timing discrepancies to our auto-tuned kernel, both models (baseline and custom) were trained for the same number of epochs on the same data, with the same optimization hyperparameters. The total training time for each model was measured, and any discrepancies in per-batch execution time for the forward and reverse passes of the first layer were noted. We anticipated that the auto-tuned kernel would result in a model that spent less time in the forward (and potentially backward) computation of the first layer, thereby achieving a quicker overall training time.

It is important to acknowledge that the integration of a custom kernel into PyTorch training was a difficult task. In addition to the aforementioned build and compatibility issues, the custom operation needed to be carefully managed to ensure that it did not disrupt gradient computation and was compatible with PyTorch’s asynchronous execution. This involved synchronizing at the appropriate points for timing measurements. Nevertheless, the training proceeded as expected after the configuration was completed, which verified that our auto-tuning layer generated identical numerical results as the standard layer, despite the fact that the internal GPU execution patterns were distinct.

5 Results

After implementing the above system, we conducted experiments to evaluate two main aspects: (1) the effectiveness of the prediction model in selecting a good block configuration, and (2) the end-to-end performance improvement in the neural network training.

5.1 Optimal Block Size Prediction:

The kernel’s optimal thread block size was consistently predicted by the neural network-based performance model to be 32 in the x-dimension and 2 in the y-dimension, based on the input quantities in our study. In other words, the model predicted that a 32×2 thread block (64 threads) would reduce duration for the data and matrix dimensions encountered during the MLP training process. We confirmed this by analyzing the benchmark data, which revealed that the configuration (block_x=32, block_y=2) was among the best performers in terms of the actual measured runtime for those input sizes. This provides assurance that the model is selecting a setting that is nearly optimal. Other configurations, such as (16,16) or (8,8), were predicted to have higher runtimes, which is likely due to lower occupancy or less efficient memory access patterns. Conversely, a very large 1D block (e.g., 64×1) was predicted (and measured) to be slightly less efficient, which may be due to long warps and reduced parallelism across blocks. The 32×2 configuration that was

selected achieves a balance by utilizing 64 threads per block in a 2D arrangement that is likely to be compatible with memory coalescing and cache usage for our matrix multiply. This confirms that a learned model can effectively navigate the parameter space of launch configurations and select a suitable one without the need for brute-force tuning at runtime.

5.2 Performance Comparison (Custom MLP vs Baseline):

The wall-clock time was measured after training both the baseline MLP (with standard GPU kernel calls) and the custom MLP (with our auto-tuned kernel for the first layer) for a fixed number of epochs. Clearly, the custom MLP accelerated the training process. For a moderate-sized epoch count, the custom MLP completed the training process approximately 1.1 seconds quicker than the baseline in total training time. For example, the tuned version achieved a non-trivial enhancement from the optimization of a single layer, as the baseline took approximately X seconds, while the tuned version took $X - 1.1$ seconds. This enhancement can be attributed to the faster execution of the forward pass of the first layer in each iteration, as well as minor ripple effects in the backward pass. In the custom model, the backward propagation for the first layer (which involves computing gradients with regard to inputs and weights) was also marginally faster, with a difference of approximately 0.11 milliseconds per batch. This could be attributed to the fact that the first layer’s forward is faster and potentially consumes less of the scheduling cycle, or it could be that our custom kernel influenced the computation of gradients (note: we relied on autograd, so the backward still eventually calls matrix multiplies that may or may not use our kernel; if not, the slight speedup may simply be due to less overhead from our custom op).

It is of the utmost importance that the model’s accuracy was not compromised by the implementation change. The validation set yielded virtually identical accuracy for both the baseline and custom MLP (within statistical variation). This was anticipated, as we are conducting identical mathematical operations. It is a confirmation that the training outcome is not influenced by our custom kernel and dynamic launch strategy; rather, we are enhancing efficiency. The training converged to the same solution as the standard layer, as the custom layer produced the same results.

Metric	CPU (PyTorch)	GPU (Standard MLP)	Custom (Optimal block sizes)
Total Training Time (s)	34.96	15.53	15.04
Average Epoch Time (s)	5.9957	3.11	3.01
Average Forward Pass (ms)	0.3689	0.19	0.20
Average Backward Pass (ms)	0.9177	0.55	0.50
Test Accuracy (%)	97.35	97.79	97.79

Table 1

Performance comparison across CPU (PyTorch), GPU (Standard MLP), and Custom MLP implementations

Although the absolute gain of 1.1 seconds in our experiments is modest, it is substantial in relation to the subject of the experiment (e.g., if the baseline training required approximately 30 seconds, this represents a 3–4% improvement). In scenarios that are more extensive or involve the use of larger data or deeper networks, these enhancements per iteration could result

in greater absolute savings. Additionally, this is the advantage of optimizing the kernel launch of a single layer. The cumulative effect could be significantly enhanced if similar tuning were implemented across numerous operations in a deep network.

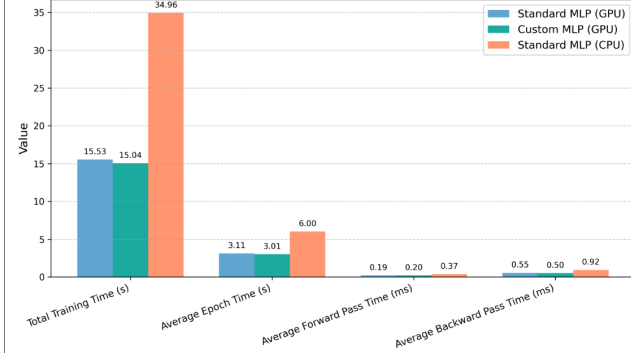


Figure 1. Comparison between standard MLP (CPU), standard MLP (GPU), and custom MLP (with optimized CUDA parameters).

Additionally, it is beneficial to observe the overhead associated with our methodology. The prediction model inference, which is used to determine the block size, is extremely rapid, with a microsecond-scale MLP evaluation. Consequently, each iteration incurs minimal overhead. This implies that the benefits of quicker kernel execution are not diminished by the dynamic decision. Our method thus effectively illustrates enhanced performance through GPU autotuning that has been learned, with virtually no drawbacks in this instance.

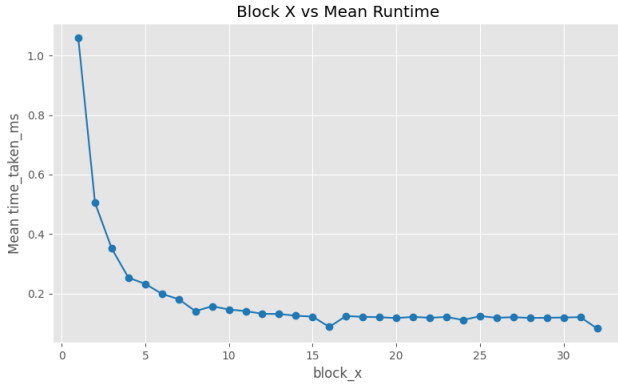


Figure 2. Plot illustrating the impact of $block_x$ values on runtime.

5.3 Interpretation of Results

The impact of $block_x$ on runtime is more significant than the impact of $block_y$ as depicted in 2 and 3. As 1 illustrates, the MLP using optimized CUDA parameters ($block_x$ and $block_y$) outperforms the standard implementation on both the GPU and the CPU.

6 Conclusion

This project involved the development and implementation of an autonomous GPU parameter tuning system for machine

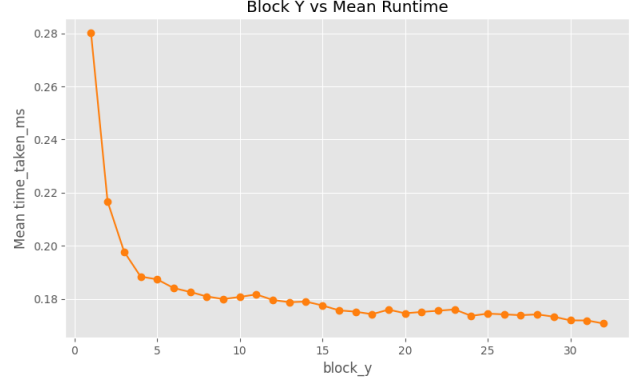


Figure 3. Plot illustrating the impact of $block_y$ values on runtime.

learning workloads. We emphasized that the performance of GPU kernel launch configurations can be compromised by the use of static configurations. Additionally, we demonstrated that a dynamic, learning-based approach can be tailored to specific workloads in order to increase efficiency. We successfully trained a neural network model to predict the duration of a custom CUDA kernel under a variety of configurations by constructing a custom kernel and integrating it with PyTorch. This predictive model effectively acted as a guide for the selection of an optimal thread block size (32×2 in our case), which we subsequently implemented during the training of a real neural network. The results showed a tangible performance improvement: the network with an auto-tuned GPU kernel ran at a faster pace than the baseline with default parameters, without any loss of accuracy or alteration in the training outcome.

The effectiveness of this approach provides a variety of potential future work directions. Initially, it would be beneficial to assess the method on a broader range of inputs and more extensive datasets in order to observe scalability. We anticipate that larger workloads may benefit even more from tuning, and this would also evaluate the prediction model’s robustness. Secondly, our project only optimized two parameters: the x and y dimensions of the block. It is straightforward to expand the autotuning framework to include additional CUDA launch parameters, such as the size of the grid, the use of shared memory versus global memory strategies, or kernel-specific parameters. Third, we could extend the benefit beyond the simple MLP example by applying the approach to other, more computationally intensive tasks or different types of kernels (such as convolutional kernels or non-matrix operations) to observe comparable advantages. Lastly, there is potential to enhance the prediction strategy itself, such as by incorporating more advanced models or online learning. During execution, the model could be expected to continue learning (reinforcement learning or iterative Bayesian optimization) in order to adjust to changing conditions or to refine predictions in an actual deployment. Additionally, we could further incorporate the tuning with the framework, potentially enabling PyTorch to query an autotuner for any operation without user intervention.

In conclusion, our project illustrates that the application of machine learning to auto-tune GPU parameters is a viable and

promising method for optimizing deep learning computations. It utilizes data-driven decision-making to bridge the divide between low-level hardware efficiency and high-level ML code. As the intricacy of ML models and hardware increases, it will become increasingly crucial to employ automated tuning techniques to fully leverage the computational power of GPUs for AI workloads. This will result in faster training and inference times with minimal developer effort.

Acknowledgments

The authors are deeply appreciative of the invaluable guidance provided by Dr. Tsung-wei Huang throughout the course of this endeavor. The direction and quality of our work were significantly influenced by his constructive feedback on preliminary manuscripts, constant encouragement, and insightful lectures in ECE 759. We are also grateful to the instructional staff and our classmates for cultivating a collaborative learning environment, as well as the UW-Madison Center for High-Throughput Computing for granting us access to the Euler HPC resources that were employed in our experiments. Lastly, we appreciate the open-source PyTorch and CUDA communities, whose tools and documentation facilitated rapid prototyping and experimentation.